Data Structure Choices — Design Analysis

1. `ScheduleServiceImpl`

1.1. `moduleMap` — `LinkedHashMap<String, StudyModule>`

`moduleMap` is built before the BFS starts and serves two purposes:

1. **Key set extraction:** `moduleMap.keySet()` is passed to `buildInDegree` and `buildDependents` so they can distinguish in-program prerequisites (edges to include) from dangling prerequisite keys that reference modules not present in the program (edges to ignore). This prevents dangling keys from inflating in-degrees and blocking modules that have no real unmet prerequisites.
2. **Deterministic ordering:** `LinkedHashMap` preserves insertion order, so the initial ready-queue seed (all keys with in-degree zero) is always produced in the same order for the same input. This makes the generated schedule deterministic: identical module sets always produce the same layering.

Note: the BFS loop itself works only with module keys (`String`) — it does not look up `StudyModule` objects from `moduleMap` during the loop. Module details are resolved later by the controller when building the schedule view, via `StudyModuleService.findModuleMapByDegreeProgramId`.

`"Math"`	StudyModule(key: `"Math"`, ects: 4, …)
`"OO"`	StudyModule(key: `"OO"`, ects: 4, …)
`"DB1"`	StudyModule(key: `"DB1"`, ects: 4, …)
⋮ (insertion order)	⋮
`"Thesis"`	StudyModule(key: `"Thesis"`, ects: 12, …)

Table 1: moduleMap — every key maps to its StudyModule

1.2. `inDegree` — `LinkedHashMap<String, Integer>`

Stores how many prerequisites each module still has left to satisfy. `LinkedHashMap` is used for the same reason as `moduleMap`: insertion order makes the initial ready-queue seed deterministic.

`"Math"`	`0`	no prerequisites → enters ready queue immediately
`"OO"`	`0`	no prerequisites → enters ready queue immediately
`"DB1"`	`1`	needs OO
`"CPI"`	`2`	needs OO, Math
`"SE1"`	`3`	needs AD1, CPI, DB1
⋮	⋮	
`"Thesis"`	`3`	needs DB2, SE2, UI2

Table 2: inDegree — count of unsatisfied in-program prerequisites; a module enters the ready queue when its count reaches 0

1.3. `dependents` — `HashMap<String, List<String>>`

Stores, for each module, the list of modules that depend on it. This is used to know which in-degrees to decrement when a module is placed. The order of decrements does not affect correctness, so a plain `HashMap` is sufficient.

`"Math"`	`["CPI"]`
`"OO"`	`["DB1", "AD1", "AD2", "CPI"]`
`"DB1"`	`["DB2", "SE1", "SE2"]`
`"AD1"`	`["AD2", "SE1", "UI1"]`
⋮	⋮
`"Thesis"`	`[]`

Table 3: dependents — reverse adjacency list; when a module is placed, each entry in its list has its inDegree decremented

1.4. `ready` — `ArrayDeque<String>`

Holds module keys that are ready to be scheduled (all prerequisites satisfied). The variable is declared as `Queue<String>` and backed by `ArrayDeque` — only the queue-side operations are used: `add` (enqueue at back), `isEmpty`, `clear`, and iteration via `List.copyOf`.

Why a queue at all? A queue models the BFS layer boundary naturally: new modules are enqueued at the back as their in-degrees reach zero, and the front-to-back snapshot at the start of each iteration captures exactly the current ready layer without mixing in modules that become ready mid-iteration.

Why `ArrayDeque` and not `LinkedList`? Both implement `Queue`, but `LinkedList` allocates a separate `Node` object for every element. `ArrayDeque` stores elements in a circular array, so elements are contiguous in memory.

front → | `"Math"` | `"OO"` | ← back

Listing 1: ready — initial state after seeding; contains all modules whose inDegree = 0

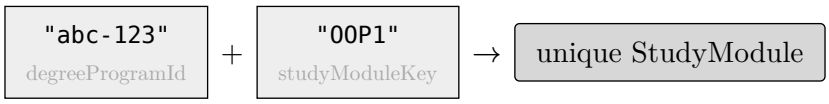
2. Repository Layer

All three repositories (`JsonDegreeProgramRepository`, `JsonStudyModuleRepository`, `JsonScheduleRepository`) extend the abstract base class `AbstractJsonRepository<T>` and persist data to JSON files in the configured data directory (`studyplanner.data.dir`).

`JsonDegreeProgramRepository` and `JsonStudyModuleRepository` both deserialize their file into a `List<T>` on every read and serialize the complete list back on every write. `JsonScheduleRepository` uses a `Map<String, Schedule>` keyed by degree-program ID.

Flat list for study modules: `JsonStudyModuleRepository` stores all study modules from all degree programs in a single flat `List<StudyModule>`. Each `StudyModule` object carries its own `degreeProgramId` field, so filtering by program is a linear scan. The alternative — one file per degree program or a nested map structure — would complicate file management (extra files to create and delete when programs are created or removed) without meaningfully improving performance at typical class sizes.

Composite business key: A study module is uniquely identified by the pair (`degreeProgramId`, `studyModuleKey`). `findStudyModuleByKey` and `saveStudyModule` both match on this composite key. This avoids the need for a synthetic surrogate ID on study modules and keeps module keys human-readable (e.g., `"OO"`, `"DB1"`), while still supporting the case where the same short key appears in two different degree programs (see Assumption A03).



Listing 2: Composite business key — degreeProgramId and studyModuleKey together uniquely identify a study module within the flat JSON list

3. Model Layer

3.1. `Schedule` and `Semester` — defensive copy + `unmodifiableList` view

Both `Schedule` and `Semester` are value objects whose lists are fully constructed before the objects are created.

- `Semester(int semesterNumber, List<String> studyModuleKeys)` — the BFS builds a `semesterKeys` list for each layer and passes it to the constructor. The constructor copies it immediately with `new ArrayList<>(studyModuleKeys)`, so the caller's list and the internal list are independent from that point on.
- `Schedule(LocalDateTime calculatedAt, List<Semester> semesters)` — likewise, the BFS accumulates semesters in a local list and passes the finished list to the constructor, which copies it with `new ArrayList<>(semesters)`.

The defensive copy ensures that the internal state cannot be modified through the reference the caller still holds.

The `getSemesters()` and `getStudyModuleKeys()` getters go one step further: they return `Collections.unmodifiableList` views of the internal lists. This prevents external code from mutating the object after construction. The unmodifiable wrapper is a zero-copy $O(1)$ operation — it creates a thin read-only view around the existing `ArrayList` without duplicating the data.



Listing 3: The constructor makes a defensive copy; getters wrap it in an unmodifiable view so neither the original caller nor any later reader can mutate the internal state

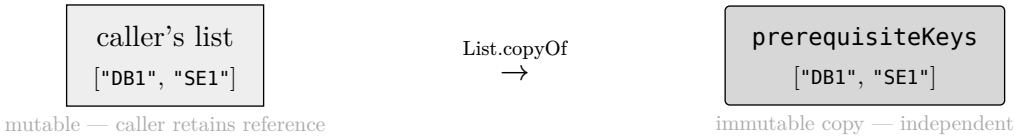
3.2. `StudyModule` prerequisite keys — `List.copyOf`

`StudyModule` is constructed inside the service layer (`StudyModuleServiceImpl`). The prerequisite keys originate from form input (collected by the controller) and are passed as a `List<String>` through the service call into the constructor. If the constructor stored that reference directly,

the caller could modify the list afterwards (accidentally or otherwise) and silently change a module it already saved.

`List.copyOf` makes a new, independent, immutable list at construction time, severing any connection to the original. An `unmodifiableList` wrapper would not be enough here: it wraps the caller's list rather than copying it, so the caller would still hold a mutable reference to the underlying data.

The key difference between the two cases: `Schedule`/`Semester` own their lists from the start and just need to hide them from callers, so a wrapper is enough. `StudyModule` receives a list it does not own, so it must copy it.



Listing 4: `List.copyOf` creates an independent immutable copy at construction time, severing any link to the caller's original list